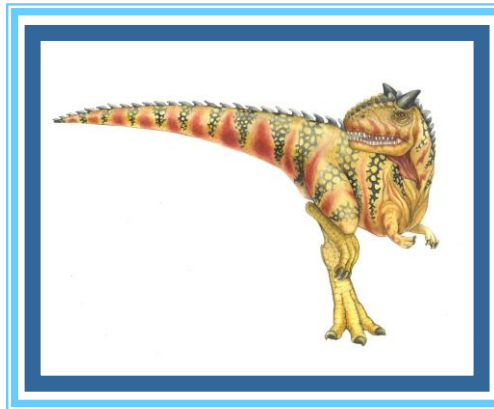


9. Bölüm: Sanal Bellek

Dr. Öğr. Üyesi Ertan Bütün

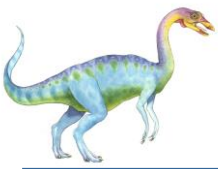




Sanal Bellek

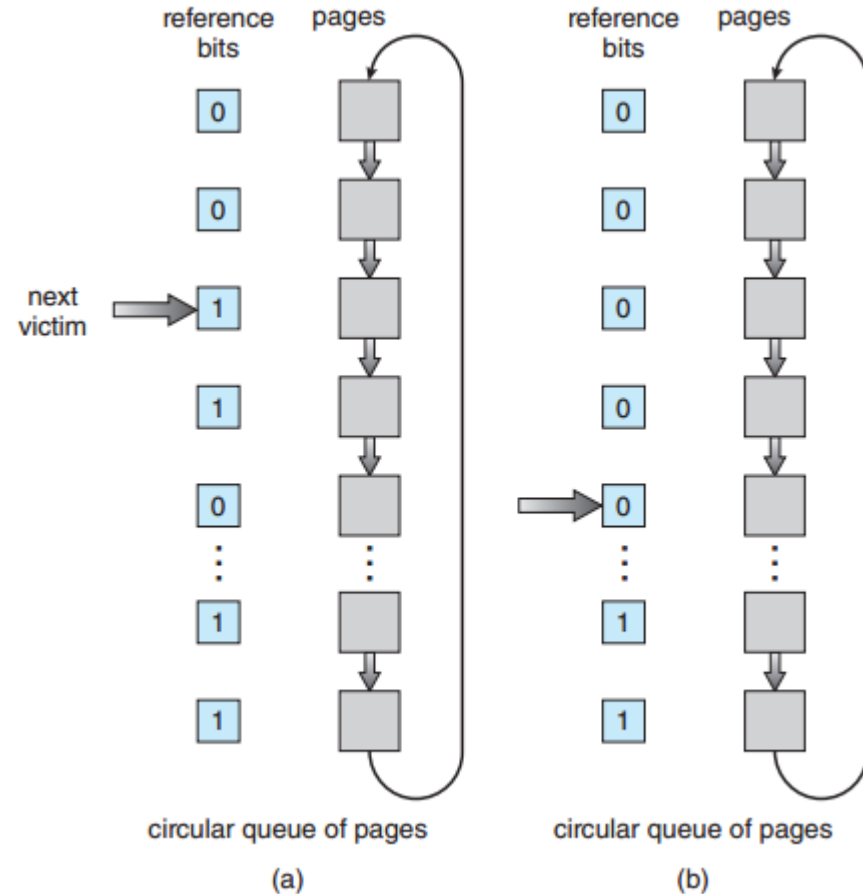
- Page Replacement
 - Basic Page Replacement
 - First-In-First-Out Algorithm
 - Optimal Page Replacement Algorithm
 - Least Recently Used Algorithm
 - LRU Approximation Algorithms
 - ▶ Additional-Reference-Bits Algorithm
 - ▶ **Second-Chance Algorithm**
 - ▶ Enhanced Second-Chance Algorithm
 - Page-Buffering Algorithms
 - Applications and Page Replacement

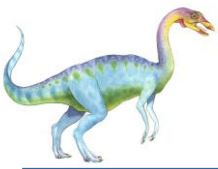




Second-chance Algorithm

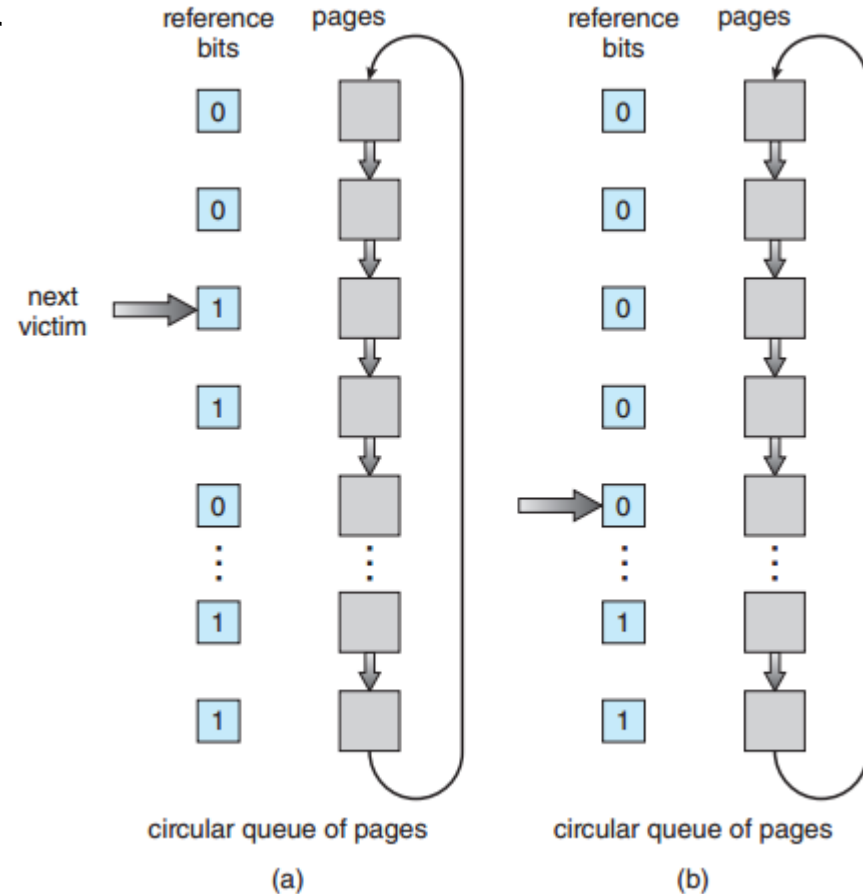
- Second-chance replacement algoritması, FIFO replacement algoritmasına benzer şekilde çalışır, ancak FIFO'ya göre seçilen page'in değiştirilmesi sırasında ikinci bir şans verir.
 - Clock algoritma olarak da isimlendirilir.
- Değiştirilecek bir page seçilirken, onun referans biti incelenir,
 - 0 ise bu page değiştirilir;
 - 1 ise page'e ikinci bir şans verilir ve bir sonraki FIFO page'ine geçilir.

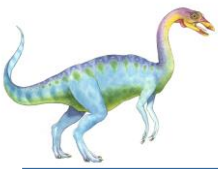




Second-chance Algorithm

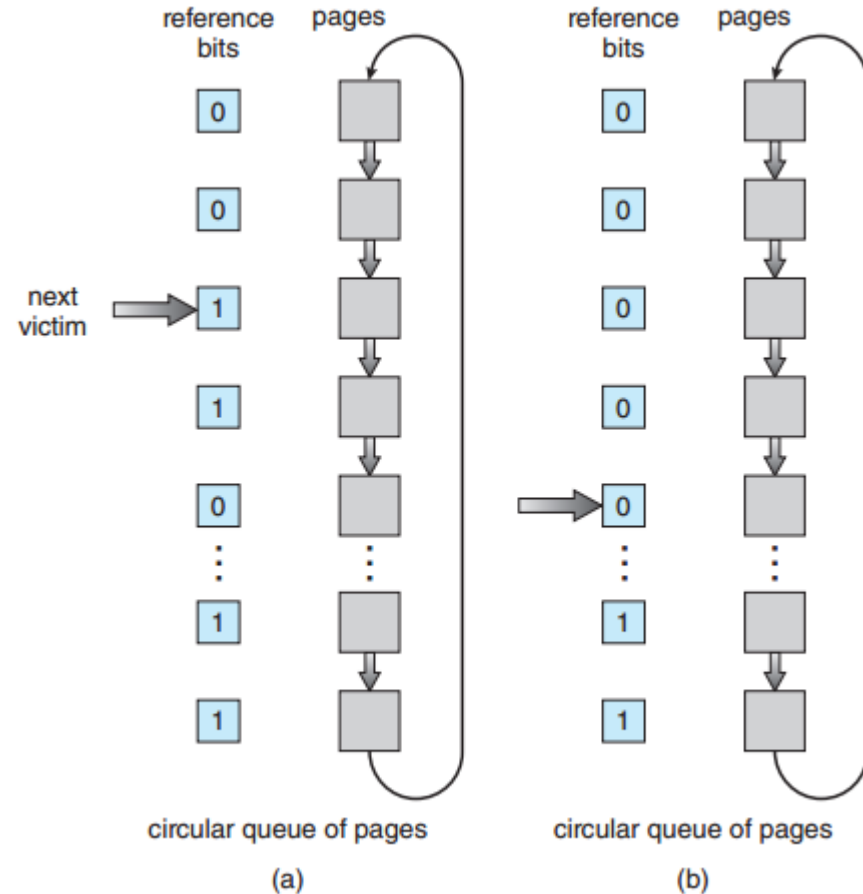
- Bellekte frame listesinde zaten var olan bir page'e tekrar başvuru, o page'e bir ödül olarak ikinci şans verilmesini sağlar, referans biti 1 yapılır.
- Bir page ikinci şansını kullandığında referans biti temizlenir ve varış zamanı mevcut zamana sıfırlanır.
 - Bu nedenle, ikinci bir şans verilen bir page, diğer tüm page'ler değiştirilene (veya ikinci bir şans verilene) kadar değiştirilmeyecektir.
 - Ayrıca, eğer bir page referans bitini korumak için yeterince sık kullanılıyorsa, asla değiştirilmeyecektir.





Second-chance Algorithm

- Second-chance algoritmasını uygulamanın bir yolu dairesel bir kuyruktur.
- Bir pointer, sonraki hangi page'in değiştirileceğini gösterir.
- Değiştirmek için bir page gerektiğinde, pointer referans biti 0 olan bir page bulana kadar ilerler.
- İlerledikçe referans bitlerini temizler.
- Bir kurban page bulunduğunda, page değiştirilir ve yeni page bu konumda dairesel kuyruğa eklenir.





Second-chance Algorithm

- Second chance algoritmasına göre şekilde verilen referans string'i için page yerleşimi nasıl olur, toplam kaç page hatası verir? (Process başına bir seferde 3 page bellekte olabilir)

Reference String: 0, 4, 1, 4, 2, 4, 3, 4, 2, 4, 0, 4, 1, 4, 2, 2, 3, 1

Total Free Frames Allocated: 3

Find Total # of page faults

0, 4, 1, 4, 2, 4, 3, 4, 2, 4, 0, 4, 1, 4, 2, 2, 3, 1

0	
1	
2	
3	
4	

Page Table





Second-chance Algorithm

Reference String: 0, 4, 1, 4, 2, 4, 3, 4, 2, 4, 0, 4, 1, 4, 2, 2, 3, 1

Total Free Frames Allocated: 3

Find Total # of page faults

✓ ✓ ✓
0, 4, 1, 4, 2, 4, 3, 4, 2, 4, 0, 4, 1, 4, 2, 2, 3, 1

Reference bit

✓ 0	0
1	0
2	0
3	0
4	0

0	0
1	4
2	1

Frames

Page Table





Second-chance Algorithm

Reference String: 0, 4, 1, 4, 2, 4, 3, 4, 2, 4, 0, 4, 1, 4, 2, 2, 3, 1

Total Free Frames Allocated: 3

Find Total # of page faults

✓ ✓ ✓ ✗
0, 4, 1, 4, 2, 4, 3, 4, 2, 4, 0, 4, 1, 4, 2, 2, 3, 1

Reference bit

0	0
1	0
2	0
3	0
4	0

0	0
1	4
2	1

Frames

Page Table





Second-chance Algorithm

Reference String: 0, 4, 1, 4, 2, 4, 3, 4, 2, 4, 0, 4, 1, 4, 2, 2, 3, 1

Total Free Frames Allocated: 3

Find Total # of page faults

0, 4, 1, 4, 2, 4, 3, 4, 2, 4, 0, 4, 1, 4, 2, 2, 3, 1

✓

	Reference bit
0	0
1	0
2	0
3	0
4	1

Page Table

0	0 (0)
1	4 (1)
2	1 (0)

Frames

Reference bits





Second-chance Algorithm

Reference String: 0, 4, 1, 4, 2, 4, 3, 4, 2, 4, 0, 4, 1, 4, 2, 2, 3, 1

Total Free Frames Allocated: 3

Find Total # of page faults = ?



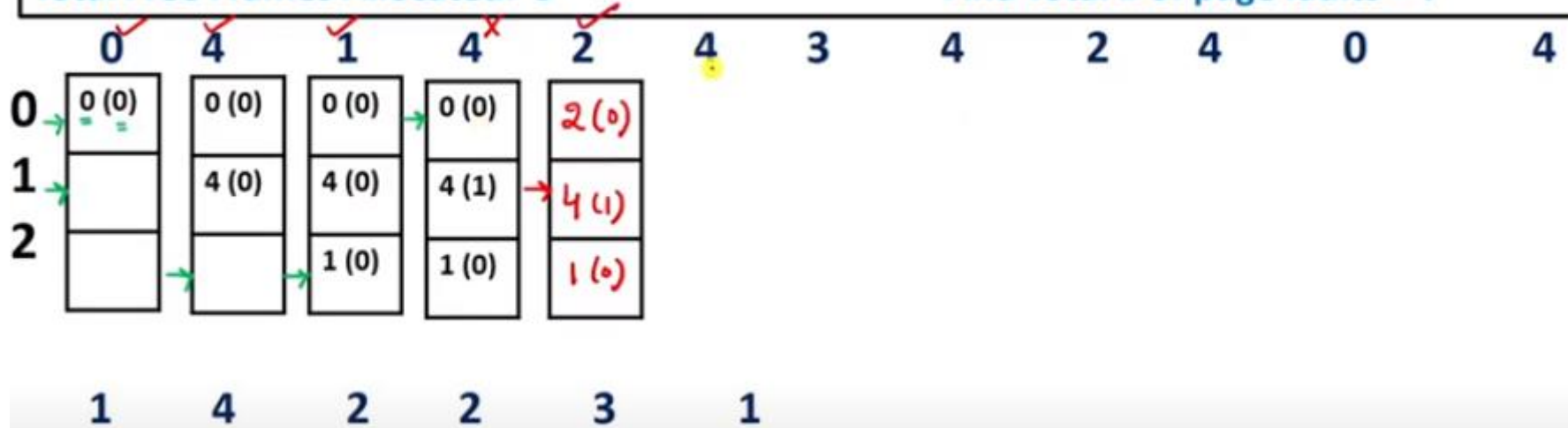


Second-chance Algorithm

Reference String: 0, 4, 1, 4, 2, 4, 3, 4, 2, 4, 0, 4, 1, 4, 2, 2, 3, 1

Total Free Frames Allocated: 3

Find Total # of page faults = ?



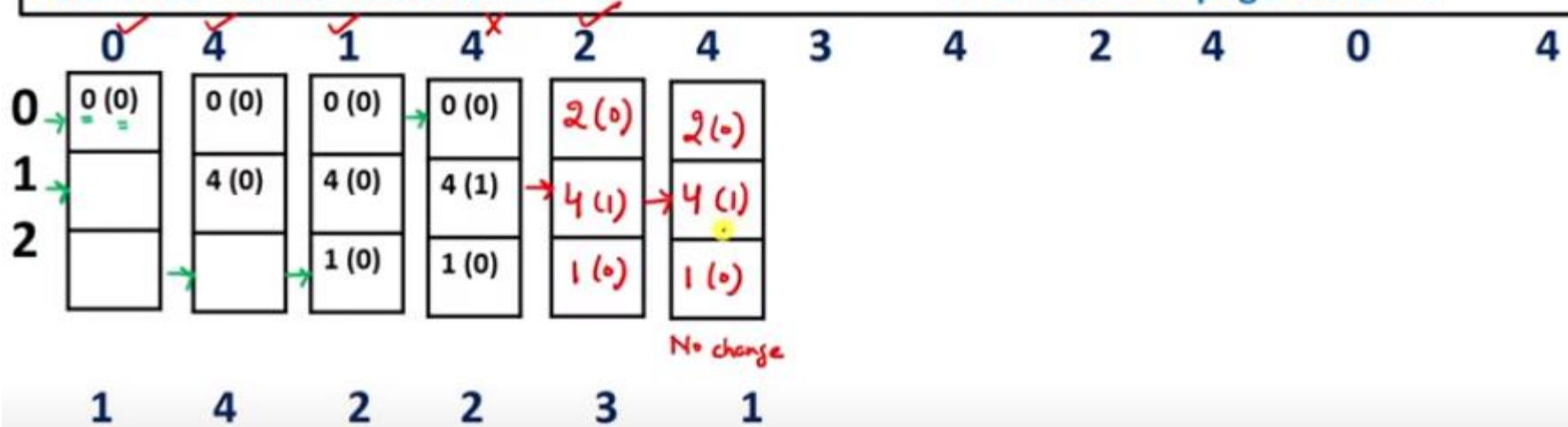


Second-chance Algorithm

Reference String: 0, 4, 1, 4, 2, 4, 3, 4, 2, 4, 0, 4, 1, 4, 2, 2, 3, 1

Total Free Frames Allocated: 3

Find Total # of page faults = ?



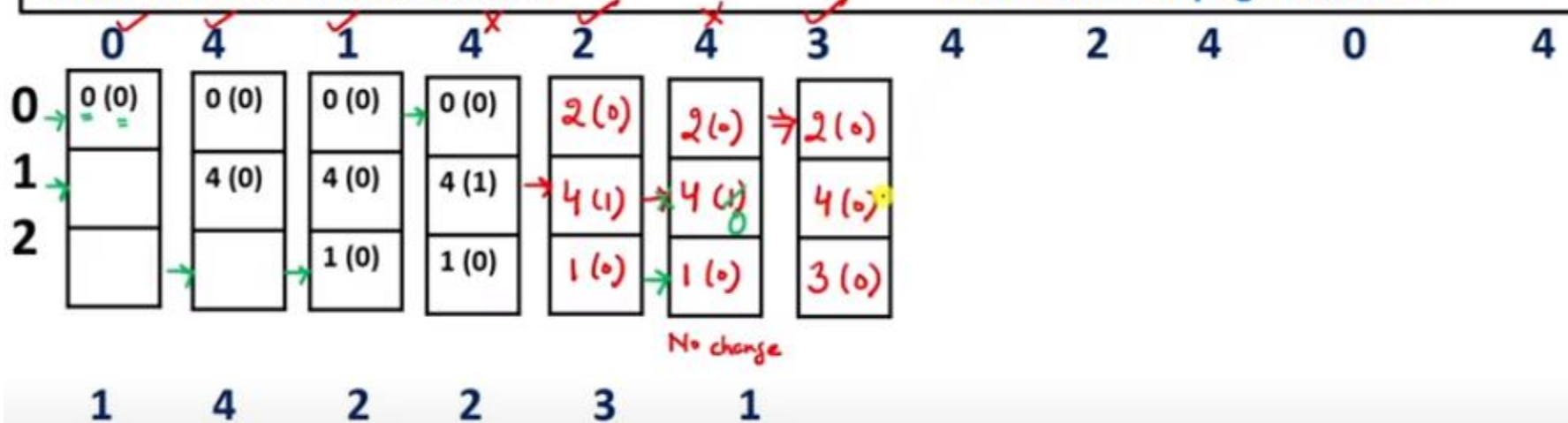


Second-chance Algorithm

Reference String: 0, 4, 1, 4, 2, 4, 3, 4, 2, 4, 0, 4, 1, 4, 2, 2, 3, 1

Total Free Frames Allocated: 3

Find Total # of page faults = ?



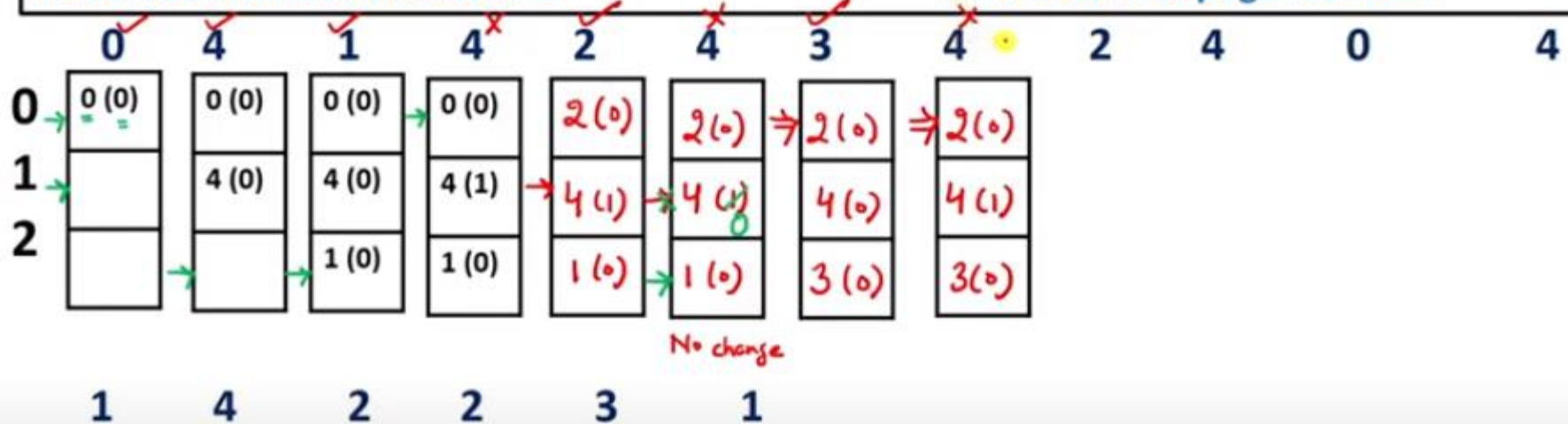


Second-chance Algorithm

Reference String: 0, 4, 1, 4, 2, 4, 3, 4, 2, 4, 0, 4, 1, 4, 2, 2, 3, 1

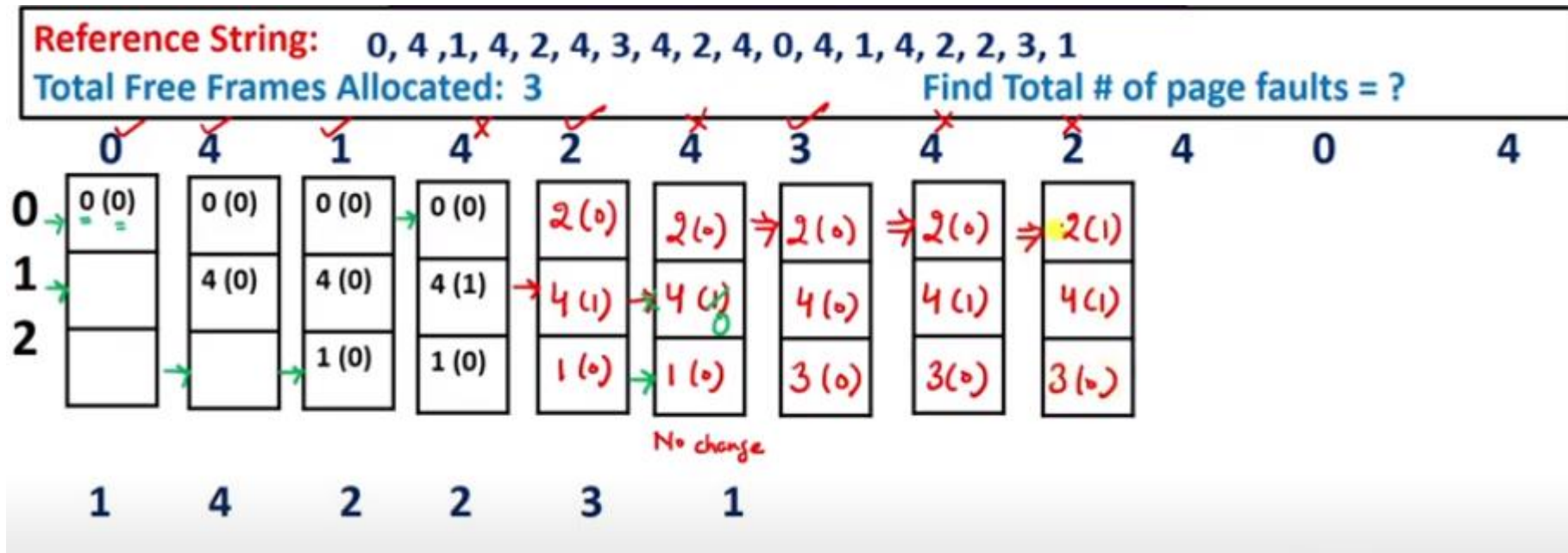
Total Free Frames Allocated: 3

Find Total # of page faults = ?





Second-chance Algorithm



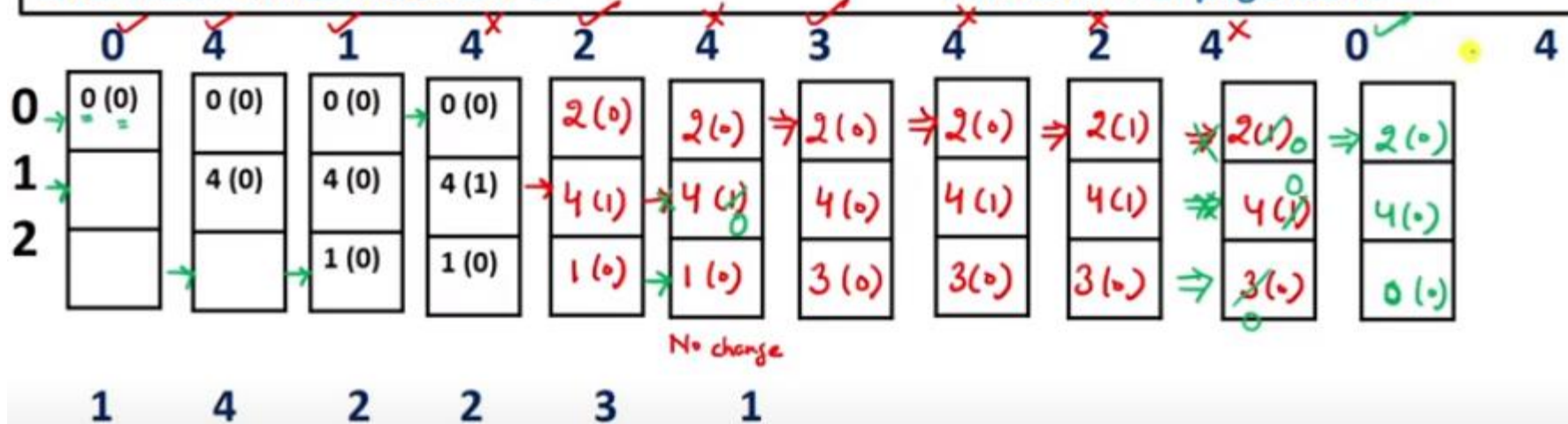


Second-chance Algorithm

Reference String: 0, 4, 1, 4, 2, 4, 3, 4, 2, 4, 0, 4, 1, 4, 2, 2, 3, 1

Total Free Frames Allocated: 3

Find Total # of page faults = ?





Find Total # of page faults = ? 10





Enhanced Second-Chance Algorithm

- Referans biti ve değişiklik (modified) bitini sıralı bir çift halinde dikkate alarak Second-Chance algoritmasını geliştirebiliriz. Bu iki bit (Referans biti, değişiklik biti) ile aşağıdaki dört olası sınıfa sahibiz:
 1. (0, 0) ne yakın zamanda kullanıldı ne de değiştirildi – değiştirilecek en iyi page
 2. (0, 1) yakın zamanda kullanılmamış ancak değiştirilmiş – o kadar iyi değil, page değiştirmeden önce ikincil depolamaya yazılmalıdır.
 3. (1, 0) yakın zamanda kullanılmış ancak değiştirilmemiş – muhtemelen yakında tekrar kullanılacak
 4. (1, 1) yakın zamanda kullanılmış ve değiştirilmiş - muhtemelen yakında tekrar kullanılacak ve değiştirmeden önce ikincil depolamaya yazılması gerekiyor
- Page'ler arasında düşük sınıf değerine sahip olan page değiştirilecek page'tir.





Enhanced Second-Chance Algorithm

- Her page bu dört sınıftan birindedir. Page değişimi istendiğinde, second chance algoritmasıyla aynı şema kullanılır; ancak page'in sadece referans bitine bakmak yerine, o page'in ait olduğu sınıfı incelenir.
- En düşük sınıf değeriyle karşılaşılan ilk page değiştirilir.
- Bu algoritma ile I/O sayısını azaltmak için hem page'in yakın zamanda kullanılıp kullanılmadığı hem de değiştirilip değiştirilmediği dikkate alınıyor.





Counting-Based Page Replacement

- Bu algorithma bir page'e yapılmış referansların bir sayacını tutulur, bu sayaçla aşağıdaki iki şema geliştirilebilir:
- **Least Frequently Used (LFU) Algorithm:** en az sayıda kullanılan page değiştirilir, aktif olarak kullanılan bir page büyük bir referans sayısına sahiptir.
- **Most Frequently Used (MFU) Algorithm:** En düşük sayaç değerine sahip page'in muhtemelen yeni getirildiği ve henüz kullanılmadığı düşünülür. Dolayısıyla en çok sayıda kullanılan page değiştirilir.
- Bu iki algoritmanın kullanımı yaygın değildir, uygulanması pahalıdır.





Page-Buffering Algorithms

- Belirli bir page replacement algoritmasına ek olarak genellikle başka prosedürler de kullanılır.
 - Örneğin, sistemler genellikle bir boş frame havuzu tutar. Bir page hatası oluştuğunda, önceden olduğu gibi boşaltılacak (kurban) frame seçilir.
 - Bununla birlikte, istenen page, kurban frame bellekten diske yazılmadan önce havuzdan boş bir frame'e yerleştirilir.
 - Bu prosedür, kurban frame'in page'e yazılmasını beklemeden sürecin mümkün olan en kısa sürede yeniden başlamasına izin verir. Kurban frame daha sonra yazıldığında, boşalan frame boş frame havuzuna eklenir.





Page-Buffering Algorithms

- Önceki slayttaki örneğin genişletilmiş hali olarak değiştirilmiş page'lerin bir listesi tutulabilir.
 - **Diske yazma aygıtı boшта kaldığında**, değiştirilmiş bir page seçilir ve diske yazılır. Daha sonra değiştirme biti sıfırlanır (bellekteki ile disktekinin aynı olduğunu göstermek için).
 - Bu şema, değiştirilmek üzere bir page'in seçilmesi gerektiğinde page'in diskteki hali ile güncel olma olasılığını artırır
 - Diske yazılmasına gerek kalmaz, zaten en son değiştirilmiş hali diske yazılmıştı.
- Başka bir varyasyon: bir boş frame havuzu tutulur, ancak her frame'de hangi page'in olduğu bilgisi de tutulur.
 - Frame'in bellekten diske yazılması gerektiğinde, diskten belleğe getirilecek page'in içeriği zaten boş frame havuzunda ilgili bir frame ile aynı olduğundan değiştirmeye gerek kalmadan doğrudan yeniden kullanılabilir.
 - Bu durumda I/O gerekmez. Bir page hatası oluştuğunda, öncelikle istenen page'in boş frame havuzunda olup olmadığını kontrol edilir. Değilse, boş bir frame seçilmeli ve page buraya getirilmelidir..





Applications and Page Replacement

- Bazı durumlarda, bazı uygulamaların verilere işletim sisteminin sanal belleği aracılığıyla erişmesi daha kötü performans gösterir.
- Tipik bir örnek: kendi bellek yönetimini ve I/O arabelleğini sağlayan bir veritabanıdır.
 - Bunun gibi uygulamalar, bellek kullanımını ve depolama kullanımını, genel amaçlı kullanım için algoritmalar uygulayan bir işletim sisteminden daha iyi anlar.
- Diğer bir örnek: Belleği yoğun kullanan uygulamalar page'lerin aynı anda iki kopyasının belleğe alınmasına neden olabilir.
 - İşletim sistemi page'in kopyasını I/O buffer olarak bellekte tutarken
 - Uygulama da kendi çalışması için page'i ayrıca bellekte tutabilir.





Applications and Page Replacement

- Önceki slaytta verilen sorunlar nedeniyle, bazı işletim sistemleri, özel programlara, herhangi bir dosya sistemi veri yapısı olmaksızın, büyük bir ardışık mantıksal blok dizisi olarak ikincil bir depolama bölümünü kullanma imkanını verir.
 - Bu dizi **raw disk** olarak da adlandırılır ve bu blok dizisine giriş çıkış raw I/O olarak adlandırılır.
 - Raw I/O; dosya I/O demand paging, dosya kilitleme, önceden getirme, gibi tüm dosya sistemi hizmetlerini atlar.
 - Bazı uygulamaların kendi özel amaçlı depolama hizmetlerini raw bir bölüm üzerinde uygularken daha verimli olmasına rağmen,
 - ▶ Çoğu uygulama da normal dosya sistemi hizmetlerini kullandıklarında daha iyi performans göstermektedir.





Sanal Bellek

- Background
- Demand Paging
- Copy-on-Write
- Page Replacement
- Allocation of Frames
- Thrashing





Allocation of Frames

- Process'lere frame tahsisi minimum sayıda yapılmalıdır. Bunun bir nedeni performansla ilgilidir.
- Ancak her process'e ayrılan frame sayısı azaldıkça, page hatası oranı artar, bu da process'in yürütülmesini yavaşlatır.
- Dolayısıyla herhangi tek bir komutun başvurabileceği tüm farklı page'leri tutmaya yetecek kadar frame'e sahip olmamız gerekir.
- Process başına minimum frame sayısı mimari tarafından tanımlanırken, maksimum sayı kullanılabilir fiziksel bellek miktarı ile tanımlanır.





Fixed Allocation

- En kolay yöntem olarak m adet frame n process arasında eşit olarak dağıtılır (**equal allocation**) (Her process m/n frame alır).
- Farklı process'ler farklı sayıda frame'e ihtiyaç duyarlar. Her process'e atanacak frame sayısı process'in boyutuna göre belirlenebilir (**proportional allocation**).
- S tüm process'lerin toplam boyutu, s_i i . process'in boyutu olsun:

$$S = \sum s_i$$

- s_i process'ine atanacak frame sayısı a_i , şöyle gibi hesaplanır:

$$a_i = (s_i / S) \times m$$

- $m = 62$ frame, $s_1 = 10$ ve $s_2 = 127$ page olsun, s_1 ve s_2 process'lerine ayrılacak frame'ler:

$$a_1 = (10/137) * 62 = 4$$

$$a_2 = (127/137) * 62 = 57$$





Global vs. Local Allocation

- Frame'lerin çeşitli process'lere atanma biçimindeki bir diğer önemli faktör page replacement'dır.
- Frame'ler için rekabet içinde olan process'ler açısından page replacement algoritmalarını iki geniş kategoride sınıflandırabiliriz:
- **Global replacement** - tüm frame'ler kümesinden yerdeğiştirme için bir frame seçilmesine izin verir, frame başka bir process'e tahsis edilmiş olsa bile bir process diğerinden bir frame alabilir.
 - Bir sorun, bir process için bellekteki page kümesinin yalnızca o process'e değil, aynı zamanda diğer process'lerin paging davranışına da bağlı olmasıdır.
 - ▶ Bu nedenle, tamamen dış koşullar nedeniyle aynı process oldukça farklı yürütme sürelerinde çalışabilir.
 - Genellikle daha yüksek sistem throughput sağlar, kullanımı daha yaygındır.





Global vs. Local Allocation

- **Local replacement** - her process'in yalnızca kendine tahsis edilmiş frame kümesinden seçim yapılmasına izin verilir.
- Process başına daha tutarlı performans sağlar.
- Ancak daha az kullanılan bellek page'lerini kullanıma sunmayarak belleğin verimsiz kullanılmasına neden olabilir.

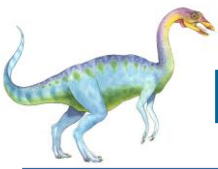




Page'lerin Geri Alınması (Reclaiming Pages)

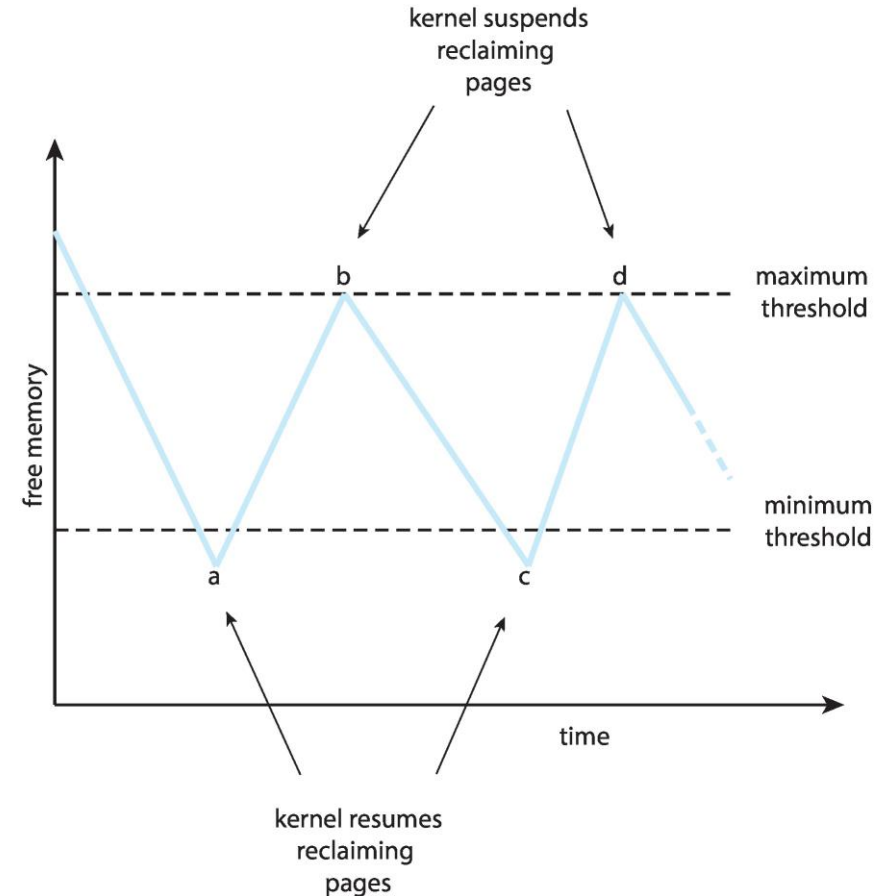
- Global page-replacement'i gerçekleştirmek için bir strateji:
 - Page replacement tetiklemesi için boş frame listesinin sıfıra düşmesi beklenmez, boş frame listesi belli bir eşik değerinde tutulur.
 - Tüm bellek istekleri boş frame listesinden karşılanır. Liste belirli bir eşiğin altına düştüğünde page replacement tetiklenir.
 - Bu strateji, yeni istekleri karşılamak için her zaman yeterli boş bellek olmasını sağlamaya çalışır.





Page'lerin Geri Alınması (Reclaiming Pages)

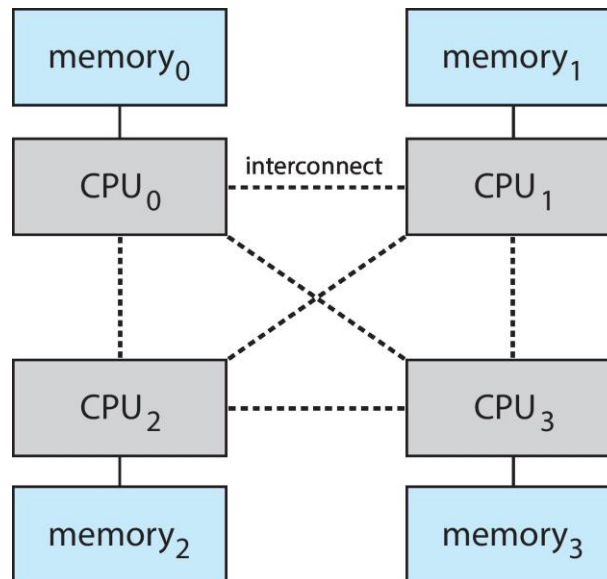
- Şekilde a noktasında boş bellek miktarının minimum eşiğin altına düştüğünde:
 - İşletim sistemi çekirdeği, page'leri geri almaya (**reclaiming**) ve bunları boş frame listesine eklemeye başlar.
 - Bu işlem maksimum eşiğe ulaşılan kadar devam eder (b noktası).
 - Page'leri geri alma, boş bellek miktarı maksimum eşiğe ulaştığında askıya alınır.





Non-Uniform Memory Access

- Şimdiye kadar sanal bellek kapsamında, tüm ana belleğin eşit oluşturulduğu veya ana belleğe eşit sürede erişildiğini varsayılmıştı.
- Birden çok CPU'ya sahip tekdüze olmayan bellek erişimi (NUMA) sistemlerinde durum farklıdır.
- Bu sistemlerde, belirli bir CPU, ana belleğin bazı bölümlerine diğerlerine erişebildiğinden daha hızlı erişebilir.
- Bu performans farklılıkları, sistemde CPU'ların ve belleğin nasıl birbirine bağlı olduğundan kaynaklanır.





Non-Uniform Memory Access

- Hangi konumlarda hangi page frame'lerinin depolandığını yönetmek NUMA sistemdeki performansı önemli ölçüde etkileyebilir.
- Amaç, process'in çalıştığı CPU'ya “mümkün olduğunca yakın” tahsis edilmiş bellek frame'lerine sahip olmaktır.
- Bu nedenle, bir process bir page hatasına neden olduğunda,
 - NUMA uyumlu bir sanal bellek sistemi, bu process'e, process'in çalıştığı CPU'ya mümkün olduğunca yakın bir frame tahsis edecektir.





Non-Uniform Memory Access

- Çok sayıda thread'e sahip bir process'in thread'lerinin farklı sistem kartlarına schedule edildiği durumlarda ise bellek tahsisi daha karmaşıktır.
 - Bu durum Solaris tarafından **lggroups (locality groups)** oluşturularak çözülmüştür.
 - Her bir lggroup, CPU'ları ve belleği bir araya toplar ve bu gruptaki her bir CPU, tanımlanmış bir gecikme aralığı içinde gruptaki herhangi bir belleğe erişebilir.
 - Ayrıca, gruplar arasındaki gecikme miktarına bağlı olarak bir grup hiyerarşisi vardır.
 - Solaris, bir process'in tüm thread'lerini bir group için schedule etmeye çalışır, process'e bellek tahsinin tümünü de yine bir grup içinde yapmaya çalışır.
 - Bu mümkün değilse, ihtiyaç duyulan kaynakların geri kalanı için yakındaki grupları seçer.





Sanal Bellek

- Background
- Demand Paging
- Copy-on-Write
- Page Replacement
- Allocation of Frames
- Thrashing





Trashing

- Bir process yürütülürken yeni bir aşamaya girdiğini ve daha fazla frame'e ihtiyaç duyduğunu varsayalım.
 - Process, page hatası yapmaya ve diğer process'lerden frame almaya başlar. Ancak bu process'ler de diğer process'lerden frame'ler alarak page hatası oluşmasına neden olurlar.
 - Page hatası alan process'ler, page'leri bellekten diske ve diskten belleğe almak için paging aygıtını kullanmalıdır. Paging aygıtı için sıraya girdiklerinde, hazır kuyruk (ready queue) boşalır.
 - Process'ler paging aygıtını beklerken, CPU kullanımı azalır. CPU scheduler, azalan CPU kullanımını görür ve multiprogramming derecesini artırır.

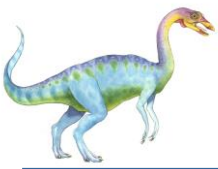




Trashing

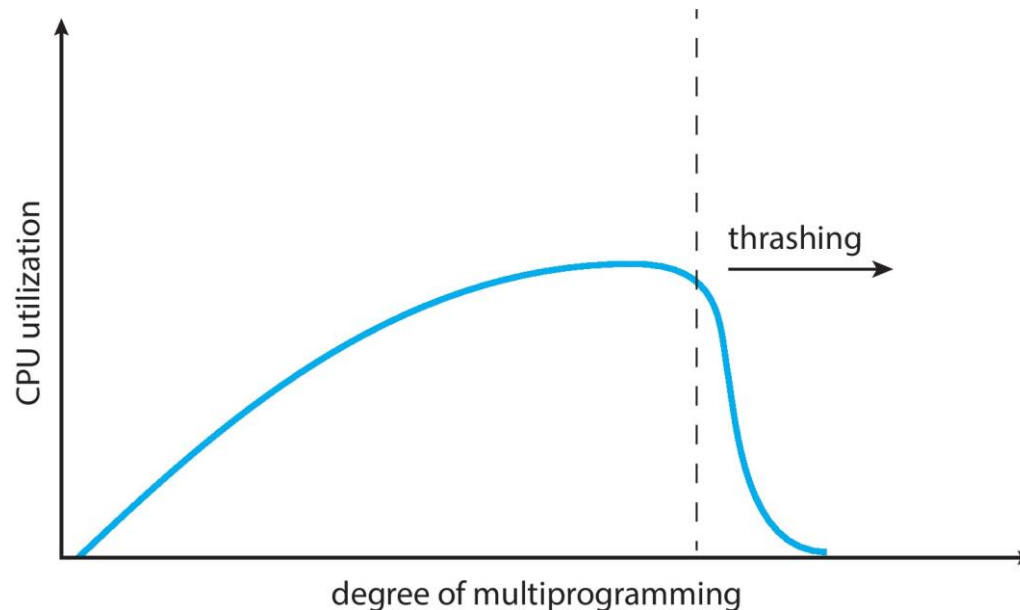
- Yeni process, yürütülen process'lerden frame'ler alarak başlamaya çalışır, bu da daha fazla page hatasına ve paging aygıtı için daha uzun bir sıraya neden olur.
- Sonuç olarak, CPU kullanımı daha da düşer ve CPU scheduler, multiprogramming derecesini daha da artırmaya çalışır.
- Bu yüksek orandaki page etkinliğine **trashing** denir. Bir process, paging yapmak için komutlarını yürütmekten daha fazla zaman harcıyorsa **trashing olmuştur**.
- **Trashing** olunca page hatası oranı muazzam bir şekilde artar. Sonuç olarak, EAT süresi artar.
- Hiçbir iş yapılmıyordur, çünkü process'ler tüm zamanlarını page'lere ayırıyordur.





Thrashing

- Bu durum, şekilde CPU kullanımının multiprocessing derecesine göre grafikte gösterilmiş.
- Multiprogramming derecesi arttıkça, CPU kullanımı da daha yavaş olmakla birlikte maksimuma ulaşılan kadar artar.
- Multiprogramming derecesi daha da artırılırsa, thrashing olur ve CPU kullanımı keskin bir şekilde düşer.
- Bu noktada, CPU kullanımını artırmak ve thrashing'i durdurmak için multiprocessing derecesi düşürülmelidir.





Trashing

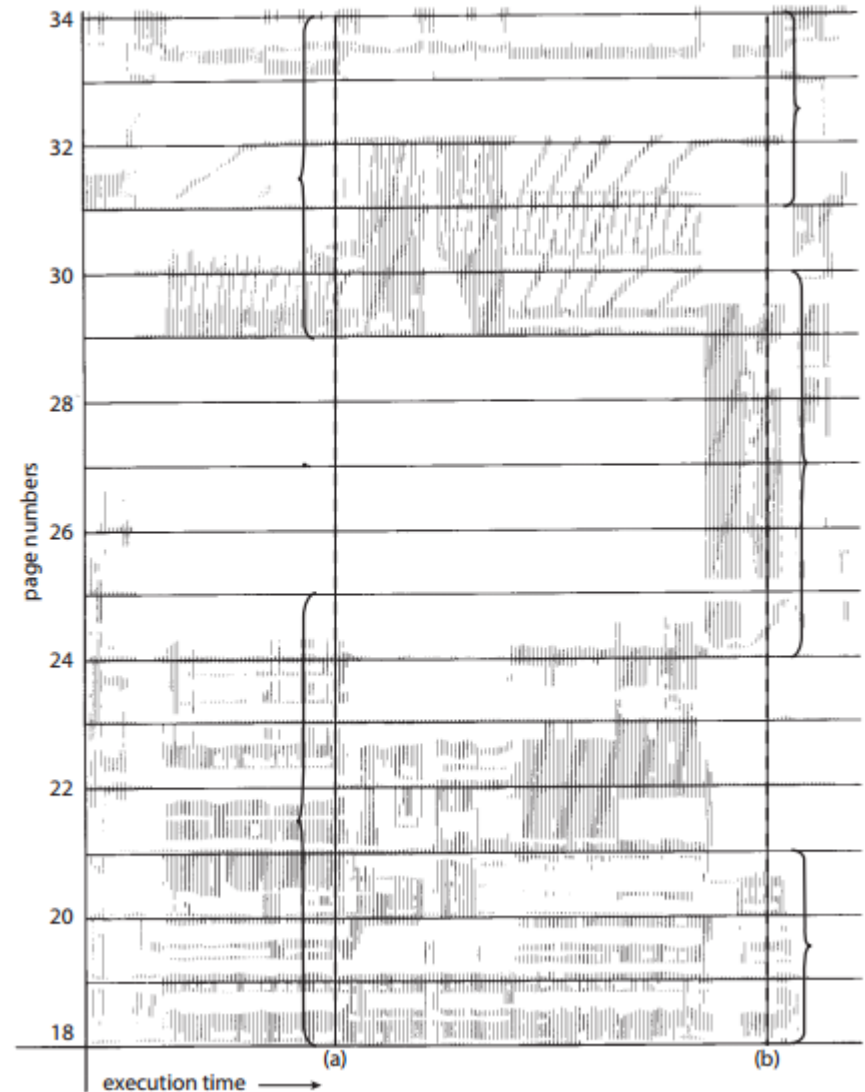
- Thrashing'i önlemek için, process'lere ihtiyaç duyduğu kadar çok frame sağlanmalıdır.
- Bir strateji olarak bir process'in gerçekte kaç frame kullandığına bakılabilir. Bu yaklaşım, process yürütmenin bölgesellik modelini (**locality model**) tanımlar.
- **Locality model**, bir process yürütüldüğünde, bir bölgeden başka bir bölgeye doğru hareket ettiğini belirtir. Bir bölge, birlikte aktif olarak kullanılan bir dizi page'dir.
- Yürütülen bir program genellikle kesişebilen birkaç farklı bölgeden oluşur.
 - Örneğin, bir fonksiyon çağrıldığında, yeni bir bölge tanımlar.
 - Fonksiyondan çıkıldığında, fonksiyonun yerel değişkenleri ve komutları artık aktif kullanımda olmadığından, process bu bölgeyi terk eder.
 - Bu bölgeye daha sonra tekrar dönülebilir.

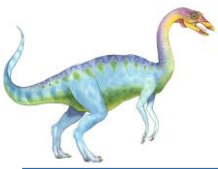




Locality In A Memory-Reference Pattern

- Şekilde, bölgeselik kavramı ve bir processin bölgeselliğinin zaman içinde nasıl değiştiği gösterilmektedir.
- (a) zamanında bölgesel page kümesi {18, 19, 20, 21, 22, 23, 24, 29, 30, 33}.
- (b) zamanında {18, 19, 20, 24, 25, 26, 27, 28, 29, 31, 32, 33}
- Bazı page'ler (örneğin, 18, 19 ve 20) her iki bölgede ortaktır.*



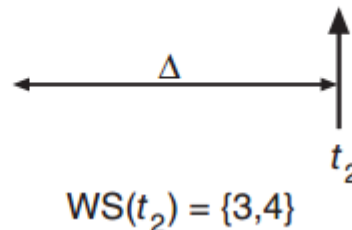
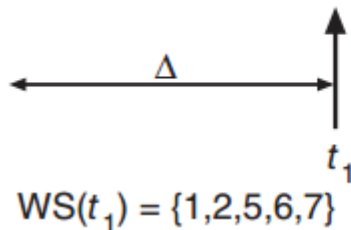


Working-Set Model

- Çalışma kümesi (**working-set**) modeli, bölgesellik varsayımına dayanmaktadır. Bu model, çalışma kümesi penceresini (working-set window) tanımlamak için Δ parametresini kullanır.
- Buradaki fikir, en son page referanslarını incelemektir. En son page referanslarındaki page grubu, working-set'tir, şekilde bunun bir örneği verilmiştir.
- Bir page aktif kullanımdaysa, working-set'te olacaktır. Son referansından sonra bir working-set window boyunca kullanılmamışsa working-set'ten çıkacaktır.
- Şekildeki örnekte $\Delta = 10$ bellek referansı olursa, t_1 anında working set $\{1, 2, 5, 6, 7\}$, t_2 anında $\{3, 4\}$ olur.

page reference table

... 2 6 1 5 7 7 7 7 5 1 6 2 3 4 1 2 3 4 4 4 3 4 3 4 4 4 1 3 2 3 4 4 4 3 4 4 4 ...





Working-Set Model

- working-set'in doğruluğu Δ seçimine bağlıdır.
 - Δ çok küçükse, tüm bölgeyi kapsamaz;
 - Δ çok büyükse, birkaç bölgeyle çakışabilir.
 - Δ sonsuz ise, working-set, işlemin yürütülmesi sırasında kullanılan page'lerin tamamından oluşan kümedir.
- Dolayısıyla working-set'in en önemli özelliği boyutudur. Sistemdeki her process için working-set size (WSS_i) hesaplarsak,
$$D = \sum WSS_i$$

D'yi toplam talep frame'i olarak düşünebiliriz.
- Her process, working-set'teki page'leri aktif olarak kullandığından i process'inin WSS_i frame'e ihtiyacı vardır.





Working-Set Model

- Toplam talep, mevcut frame'lerin toplam sayısından ($D > m$) büyükse, bazı process'lerin yeterli frame'i olmayacağından, thrashing meydana gelecektir.
- İşletim sistemi, her process'in working-set'ini izler ve bu working-set'e, working-set boyutu kadar yeterli frame tahsis eder.
- Yeterince fazla frame varsa, başka bir process başlatılabilir.
- working-set boyutlarının toplamı, kullanılabilir toplam frame sayısını aşacak şekilde artarsa, işletim sistemi askıya almak için bir process seçer.
 - Process'in page'leri swap out yapılır ve frame'leri diğer process'lere yeniden tahsis edilir. Askıya alınan process daha sonra yeniden başlatılabilir.
- working-set stratejisi, multiprogramming derecesini olabildiğince yüksek tutarken kesişmeyi önler. Böylece CPU kullanımını optimize eder.

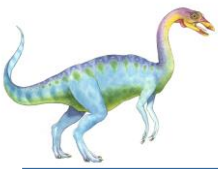




Page-Fault Frequency

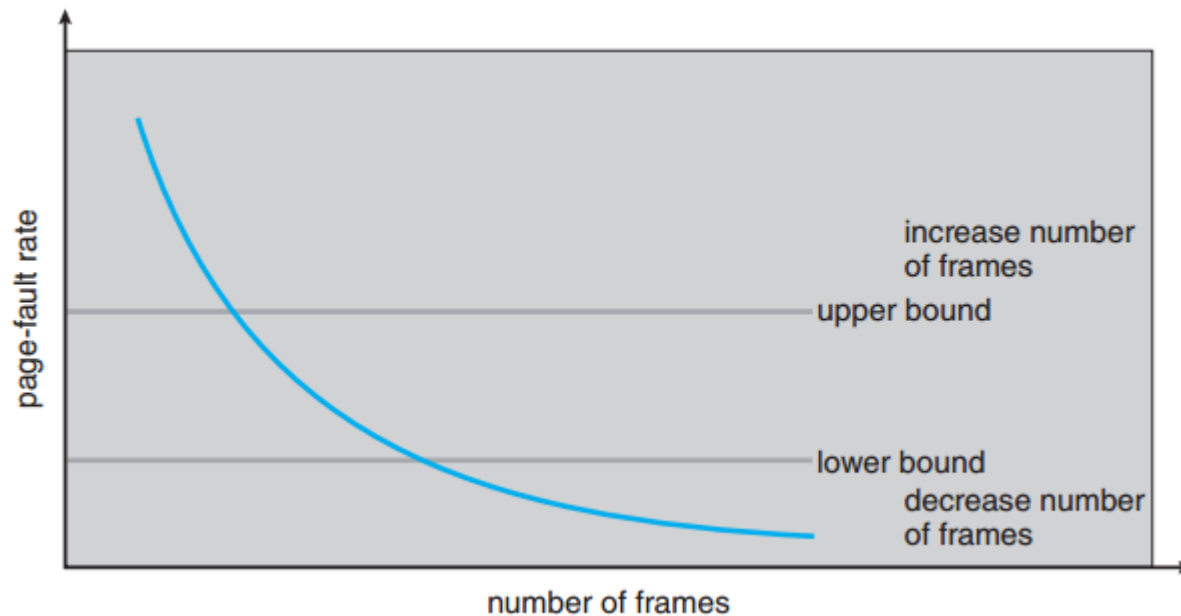
- Page hatası sıklığı (Page-Fault Frequency, PFF) stratejisi, thrashing'i kontrol etmek için working-set modeline göre daha doğrudan bir yaklaşım benimser.
- Problem, thrashing'in nasıl önleneceğidir. Thrashing, yüksek bir page hatası oranına sahiptir. Bu nedenle, page hata oranı kontrol etmek istenir.
- Page hatası çok yüksek olduğunda, process'in daha fazla frame'e ihtiyaç duyduğunu biliriz. Page hatası oranı çok düşükse, process'in çok fazla frame'i olabilir.





Page-Fault Frequency

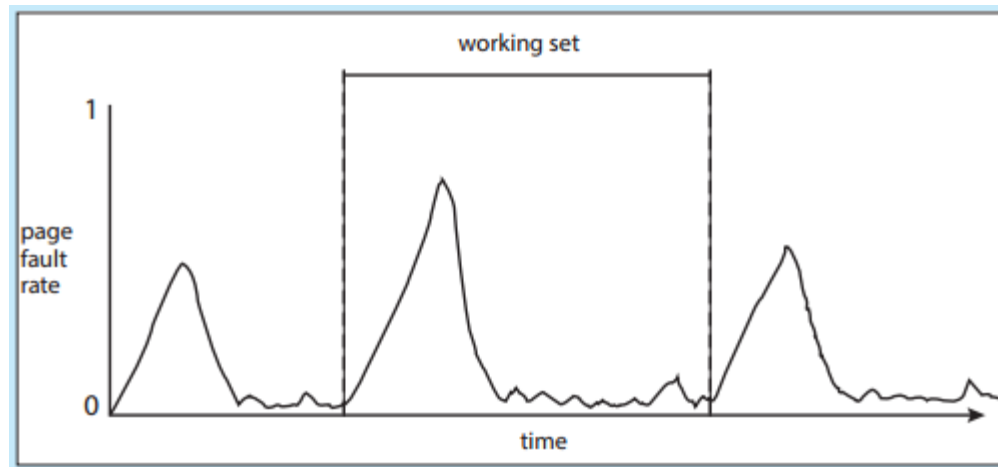
- İstenilen page hatası oranında üst ve alt sınırlar oluşturulur.
- Mevcut page hatası oranı üst sınırı aşarsa, process'e başka bir frame tahsis ederiz.
- Page hatası oranı alt sınırın altına düşerse, process'den bir frame kaldırırız.
- Böylece, thrashing'i önlemek için page hatası oranını doğrudan ölçebilir ve kontrol edebiliriz.





Working Sets and Page Fault Rates

- Bir process'in working-set ile page hata oranı arasında doğrudan bir ilişki vardır.
- Tipik olarak verilere ve kod bölümlerine yapılan referanslar bir bölgeden diğerine geçtikçe, bir process'in working-set'i zamanla değişir.
- Bir process'in working-set'ini depolamak için yeterli bellek olduğunu varsayarsak (process thrashing olmuyorsa), process'in page hata oranı zamanla tepe noktaları ve düşük değerler arasında geçiş yapacaktır.
- Bu genel davranış şekilde gösterilmiştir:*





Thrashing – Mevcut Uygulama

- Pratikte; thrashing ve sonucunda ortaya çıkan swapping, performans üzerinde ciddi bir düşüşe neden olabilir.
- Bir bilgisayar sisteminin uygulanmasında mevcut en iyi yaklaşım, mümkünse thrashing ve swapping'ten kaçınmak için yeterli fiziksel bellek eklemektir.
- Akıllı telefonlardan büyük sunuculara kadar, ekstra koşullar dışında, tüm working set'leri aynı anda bellekte tutmak için yeterli belleğin sağlanması, en iyi kullanıcı deneyimini sağlar.

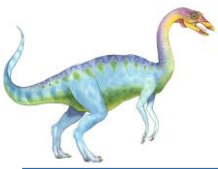




Operating System Examples

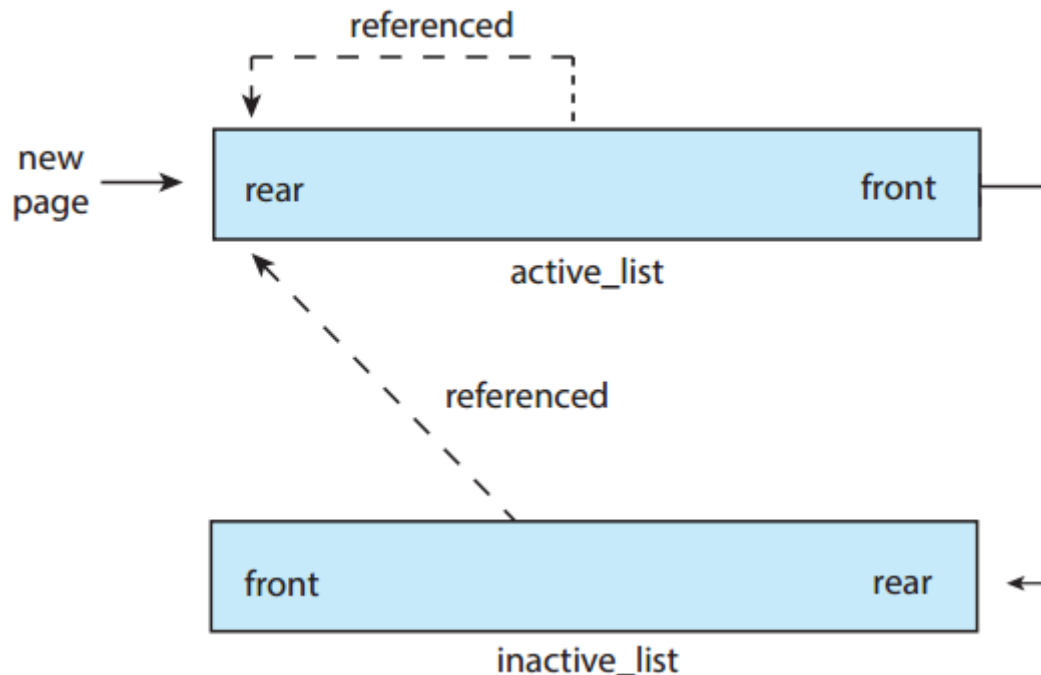
- Linux
- Windows

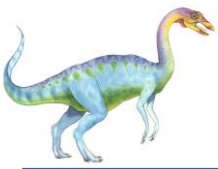




Linux

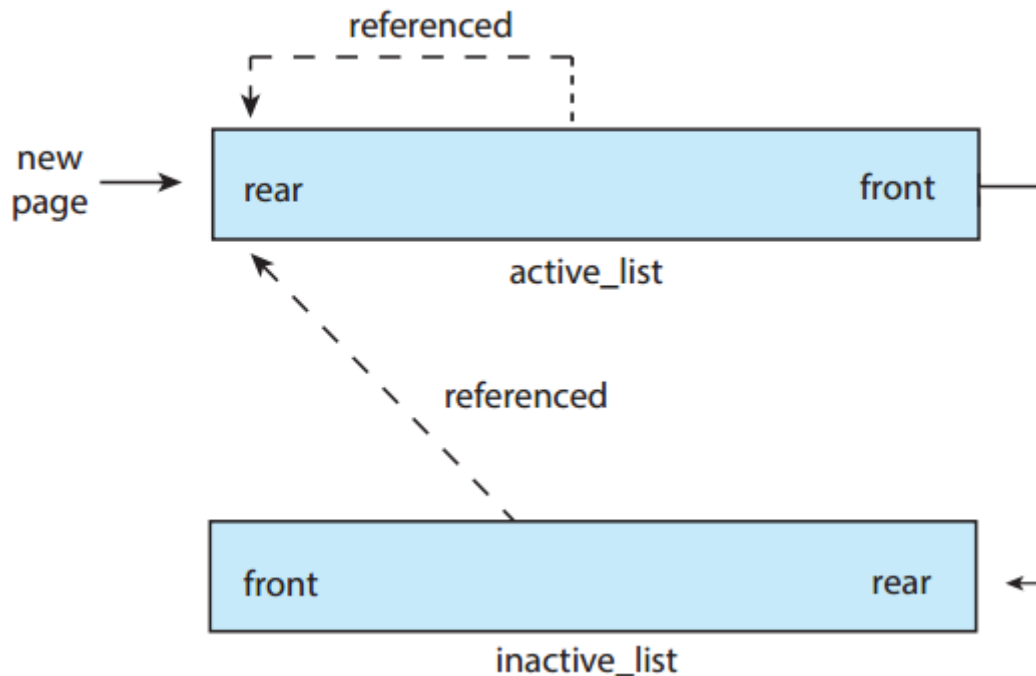
- Linux, page'leri boş frame listesinden ayırarak bir demand paging kullanır. Ek olarak, Second chance (clock) algoritmasına benzer bir genel page değiştirme politikası kullanır.
- Belleği yönetmek için Linux iki tür page listesi tutar: bir **active_list** ve bir **inactive_list**.
 - **active_list**, kullanımda olduğu düşünülen page'leri içerirken **inactive_list**, yakın zamanda başvurulmamış ve geri alınmaya uygun page'leri içerir.





Linux

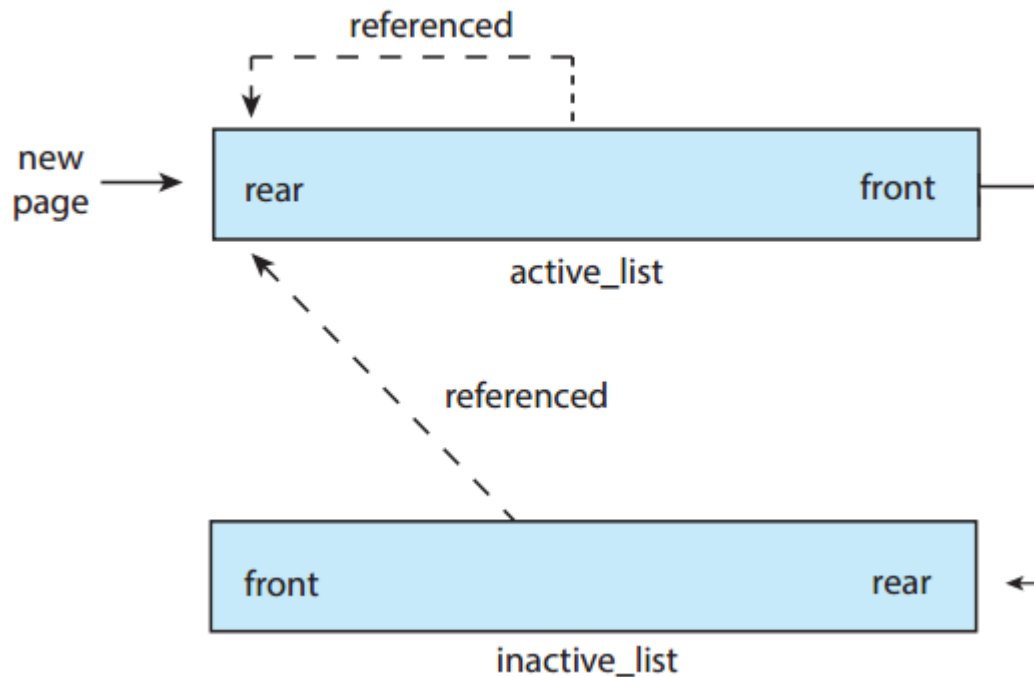
- Her page'in, page'e her başvuru yapıldığında 1 olarak set edilen bir **accessed** biti vardır. Bir page ilk tahsis edildiğinde, **accessed** biti 1 yapılır ve aktif listenin arkasına (rear) eklenir.
- Benzer şekilde, **active_list** içindeki bir page'e başvurulduğunda, onun **accessed** biti 1 yapılır ve page listenin arkasına taşınır. **active_list**'teki page'ler için **accessed** bitler periyodik olarak sıfırlanır.





Linux

- Zamanla, en az kullanılan page **active_list**'in önünde (front) olacaktır. Buradan, **inactive_list**'in arkasına geçebilir. **inactive_list**'teki bir page'e başvurulursa, **active_list**'in arkasına geri döner.





Linux

- İki liste göreceli olarak dengede tutulur, **active_list** **inactive_list**'ten çok daha fazla büyüdüğünde, **active_list**'in önündeki (front) page'ler **inactive_list**'e taşınır ve burada geri alınmak (reclaiming) için uygun hale gelirler.
- Boş bellek belirli bir eşiğin altına düşerse; kernel, **inactive_list**'teki page'leri taramaya ve bunları boş frame listesi için geri almaya başlar.





Windows

- Windows 10, bellek referanslarının yerelliğini tanıyan **clustering** olarak isimlendirilen bir strateji ile birlikte demand paging kullanarak sanal belleği uygular.
 - **clustering** ile yalnızca hatalı page değil, aynı zamanda hatalı page'in hemen önceki ve sonraki birkaç page de getirilerek page hatalarını ele alınır
- Windows 10'da sanal bellek yönetiminin önemli bir bileşeni, **working set** yönetimidir.
 - **working set minimum**, process'in bellekte sahip olması garanti edilen minimum page sayısıdır; yeterli bellek varsa, bir process'e, **working set maximum** sayısı kadar page atanabilir.
 - Yeterli bellek varsa, bir process **working set maximum'un** ötesine geçebilir. Benzer şekilde, bir process'e ayrılan bellek miktarı, bellek talebinin yüksek olduğu dönemlerde **minimum'un** altına düşebilir.





Windows

- Windows, local ve global page değiştirme ilkelerinin bir kombinasyonu ile Second-chance (clock) algoritması kullanır.
- Sanal bellek yöneticisi, boş page frame'lerinin bir listesini tutar. Bu listeye ilişkili olarak, yeterli boş belleğin mevcut olup olmadığını gösteren bir eşik değeri bulunur.
- **working set maximum'un** altındaki bir process için bir page hatası meydana gelirse, sanal bellek yöneticisi boş page'ler listesinden bir page ayırır.
 - **working set maximum'da** olan bir process bir page hatasına alırsa ve yeterli bellek varsa, process'e, **working set maximum'un** ötesine geçmesine izin veren boş bir page tahsis edilir.
 - Bununla birlikte, boş bellek miktarı yetersizse, kernel, yerel bir LRU page değiştirme ilkesi kullanarak değiştirilmek üzere process'in **working set'inden** bir page seçmelidir.





Windows

- Boş bellek miktarı belli bir eşiğin altına düştüğünde, sanal bellek yöneticisi, değeri eşiğin üzerindeki bir düzeye geri yüklemek için **automatic working-set trimming** olarak bilinen genel bir değiştirme taktiği kullanır.
 - **automatic working-set trimming**, process'lere ayrılan page sayısını değerlendirerek çalışır.
 - Bir process'e **working set minimum'dan** daha fazla page tahsis edilmişse, sanal bellek yöneticisi, yeterli bellek bulunana veya process **working set minimum'a** ulaşana kadar page'leri **working set'inden** kaldırılır.

